`secure_clear`

Document number: P1315R4 [latest]
Date: 2019-10-07
Author: Miguel Ojeda <miguel@ojeda.io>
Project: ISO JTC1/SC22/WG21: Programming Language C++
Audience: SG1

Abstract

Sensitive data, like passwords or keying data, should be cleared from memory as soon as they are not needed. This requires ensuring the compiler will not optimize the memory overwrite away. This proposal adds a new header, `<secure>`, containing a `secure_clear` function and a `secure_clear` function template that guarantee users that a memory area is cleared.

The problem

When manipulating sensitive data, like passwords in memory or keying data, there is a need for library and application developers to clear the memory after the data is not needed anymore [1][2][3][4], in order to minimize the time window where it is possible to capture it (e.g. ending in a core dump or probed by a malicious actor). Recent vulnerabilities (e.g. Meltdown, Spectre-class, Rowhammer-class…) have made this requirement even more prominent.

In order to ensure that the memory is cleared, the developer needs to inform the compiler that it must not optimize away the memory write, even if it can prove it has no observable behavior. In particular, for C++, extra care is needed to consider all exceptional return paths.

For instance, the following function may be vulnerable, since the compiler may optimize the `memset` call away because the `password` buffer is not read from before it goes out of scope:

```
void f()
{
    constexpr std::size_t size = 100;
    char password[size];

    // Acquire some sensitive data
    getPasswordFromUser(password, size);

    // Use it for some operations
    usePassword(password, size);

    // Attempt to clear the sensitive data
    std::memset(password, 0, size);
}
```

On top of that, `usePassword` could throw an exception so `std::memset` is never called (i.e. assume the stack is not overwritten and/or that the memory is held in the free store).

There are many ways that developers may use to try to ensure the memory is cleared as expected (i.e. avoiding the optimizer):

- Calling a function defined in another translation unit, assuming LTO/WPO is not enabled.
- Writing memory through a `volatile` pointer (e.g. `decaf_bzero` [5] from OpenSSL).
- Calling `memset` through a `volatile` function pointer (e.g. `OPENSSL_cleanse` C implementation [6]).
- Creating a dependency by writing an `extern` variable into it (e.g. `CRYPTO_malloc` implementation [7] from OpenSSL).

- Coding the memory write in assembly (e.g. `OPENSSL_cleanse` SPARC implementation [8]).
- Introducing a memory barrier (e.g. `memzero_explicit` implementation [9] from the Linux Kernel).
- Disabling specific compiler options/optimizations (e.g. `-fno-builtin-memset` [10] in GCC).

Or they may use a pre-existing solution whenever available:

- Using an operating system-provided API (e.g. `explicit_bzero` [11] from OpenBSD & FreeBSD, `SecureZeroMemory` [12] from Windows).
- Using a library function (e.g. `memzero_explicit` [13][9] from the Linux Kernel, `OPENSSL_cleanse` [14][6]).

Regardless of how it is done, none of these ways is — at the same time — portable, easy to recognize the intent (and/or `grep` for it), readily available and avoiding compiler implementation details. The topic may generate discussions in major projects on which is the best way to proceed and whether an specific approach ensures that the memory is actually cleansed (e.g. [15][16][17][18][19]). Sometimes, such a way is not effective for a particular compiler (e.g. [20]). In the worst case, bugs happen (e.g. [21][22]).

C11 (and C++17 as it was based on it) added `memset_s` (K.3.7.4.1) to give a standard solution to this problem [4][23][24]. However, it is an optional extension (Annex K) and, at the time of writing, several major compiler vendors do not define `__STDC_LIB_EXT1__` (GCC [25], Clang [26], MSVC [27], icc [28]). Therefore, in practical terms, there is no standard solution yet for C nor C++. A 2015 paper on this topic, WG14's N1967 "Field Experience With Annex K — Bounds Checking Interfaces" [29], concludes that "Annex K should be removed from the C standard".

Other languages offer similar facilities in their standard libraries or as external projects:

- The `SecureString` class in .NET Core, .NET Framework and Xamarin [30].
- The `securemem` package in Haskell [31].
- The `secstr` library in Rust [32].
- Limited private types in Ada and SPARK [33][34].

The basic solution

We can standarize current practise by providing a `secure_clear` function that takes a pointer and a size, which clears the memory (with indeterminate values) and guarantees that it won't be optimized away:

```
std::secure_clear(password, size);
```

As well as a `secure_clear` function template that takes a reference to a `TriviallyCopyable` object and clears it entirely:

```
std::secure_clear(password);
```

Note on C (WG14)

Since `secure_clear` (the plain function) would be very useful for C projects, we would like to work with WG14 (see Kona 2019 polls [35]) on getting it into C too and therefore importing it into C++. From there, we would define the function template on C++'s side.

Note that the intention here is not to discuss Annex K in its entirety (e.g. to make it mandatory). Instead, we want to focus on a specific need that projects have (securely clearing sensitive data from memory) and that is being used nowadays, as explained in the previous sections. In what form this should be done within C remains to be discussed.

For instance, a solution would be to just make `memset_s` mandatory. However:

- This implies specifying a particular value instead of leaving memory with indeterminate values, i.e. it looks like it is intended to ***set new*** values rather than ***disposing of old*** data. This point in particular was polled at Kona 2019 [35] and we were in favor of going for the latter interpretation.
- A different name for the function may also be key to emphasize the intended semantics.
- `memset_s`' interface/signature follows Annex K patterns, which may be objected to.

Further issues and improvements

While it is a good practise to ensure that the memory is cleared as soon as possible, there are other potential improvements when handling sensitive data in memory:

- Reducing the number of copies to a minimum.
- Clearing registers, caches, the entire stack frame, etc.
- Locking/pinning memory to avoid the data being swapped out to external storage (e.g. disk).
- Encrypting the memory while it is not being accessed to avoid plain-text leaks (e.g. to a log or in a core dump).
- Turning off speculative execution (or mitigating it). At the time of writing, Spectre-class vulnerabilities are still being fixed (either in software or in hardware), and new ones are coming up [36].
- Techniques related to control flow, e.g. control flow balancing (making all branches spend the same amount of time).

Most of these extra improvements require either non-portable code, operating-system-specific calls, or compiler support as well, which also makes them a good target for standardization. A subset (e.g. encryption-at-rest and locking/pinning) may be relatively easy to tackle in the near future since libraries/projects and other languages handle them already. Other improvements, however, are well in the research stage on compilers, e.g.:

- The SECURE project and GCC (stack erasure implemented as a function attribute) [37][38][39].
- Research on controlling side-effects in mainstream C compilers [40].

Further, discussing the addition of security-related features to the C++ language is rare. Therefore, this paper attempts to only spearhead the work on this area, providing access to users to the well-known "guaranteed memory clearing" function already used in the industry as explained in the previous sections.

Some other related improvements based on the basic building blocks can be also thought of, too:

- For instance, earlier revisions of this paper [41] proposed, in addition, an uncopyable class template meant to wrap a memory area on which `secure_clear` is called on destruction/move, plus some other features (explicit read/modify/write access, locking/pinning, encryption-at-rest, etc.). This wrapper is a good approach to ensure memory is cleared even when dealing with exceptions. During the LEWGI review, it was acknowledged that similar types are in use by third-parties. Some libraries and languages feature similar types. It was decided at Kona 2019 [35] to reduce the scope of the paper and only provide the basic building block, `secure_clear`.
- A type-modifier with similar semantics as the wrapper class template above was suggested during the LEWGI review.
- Dynamically-sized string-like types (e.g. `secure_string`) may be considered useful (e.g. for passwords).

As well as other related library features:

- A way to read non-echoed standard input (e.g. see the `getpass` module in Python [42]).

Proposal

This proposal adds a new header, `<secure>`, containing a `secure_clear` function and a `secure_clear` function template.

`secure_clear` function

```
namespace std {
    void secure_clear(void * data, size_t size) noexcept;
}
```

The `secure_clear` function is intended to make the contents of the memory range `[data, data + size)` impossible to recover. It can be considered equivalent to a call to `memset(data, value, size)` with indeterminate `value`s (i.e. there may even be different values, e.g. randomized). However, the compiler must guarantee the call is never optimized out unless the data was not in memory to begin with.

- To clarify: the call may be removed by the compiler if there is no actual memory involved, instead of forcing itself to use actual memory and then clearing it (which would make the call pointless and less secure to begin with). For instance, if the compiler chose to keep the data in a register because it is small enough (and its address was not taken anywhere else), then the call could be elided. In a way, there was no memory to clear to begin with, so it could be considered that it was not optimized out.
- Note: at Kona 2019 [35] it was polled whether the compiler should be free to implement further guarantees (e.g. clearing cache/registers containing it) and/or whether we should encourage them to do so. The result was neutral, so further input from experts would be needed to decide this.

An approach for the wording would be to lift the provision in [intro.abstract]p1 (i.e. the “as-if” rule) for calls to this function; thus conforming implementations need to emulate the behavior of the abstract machine and therefore produce code equivalent to having called the function, even if it has no effect on the observable behavior of the program. This is the same approach C11 takes for `memset_s`:

> *Unlike `memset`, any call to the `memset_s` function shall be evaluated strictly according to the rules of the abstract machine as described in (5.1.2.3). That is, any call to the `memset_s` function shall assume that the memory indicated by `s` and `n` may be accessible in the future and thus must contain the values indicated by `c`.*

Given this function imposes unusual restrictions/behavior, this paper was forwarded to EWG at Kona 2019 [35], and then to SG1 at Cologne 2019 [43].

`secure_clear` function template

```
namespace std {
    template <trivially_copyable T>
        requires (not pointer<T>)
    void secure_clear(T & object) noexcept;
}
```

The `secure_clear` function template is equivalent to a call to `secure_clear(addressof(object), sizeof(object))`.

The `not pointer<T>` constraint is intended to prevent unintended calls that would have cleared a pointer rather than the object it points to:

```
char buf[100];
char * buf2 = buf;

std::secure_clear(buf);                 // OK, clears the array
std::secure_clear(buf2);                // Error
std::secure_clear(buf2, sizeof(buf2)); // OK, explicit
```

Naming

Several alternatives were considered for the initial name of the proposed `secure_clear` function and function template, as well as the `secure` header name. Some were also suggested during the LEWGI review.

In general, there are many possible terms that may be used to form a name:

- `secure`
- `clear`
- `explicit`
- `zero`
- `scrub`
- `overwrite`
- `smash`
- `erase`
- `set`
- `mem` and variations
- `sensitive`
- …

Decisions regarding naming should come later on.

Example implementation

A trivial example implementation (i.e. relying on OS-specific functions) can be found at [44].

Acknowledgements

Thanks to JF Bastien and Billy O'Neal for providing guidance about the standardization process. Thanks to Ryan McDougall for presenting the paper at Kona 2019. Thanks to Graham Markall for his input regarding the SECURE project and the current state on compiler support for related features. Thanks to Martin Sebor for pointing out the SECURE project. Thanks to BSI for suggesting constraining the template to non-pointers. Thanks to everyone else that joined all the different discussions.

References

1. MSC06-C. Beware of compiler optimizations — *https://wiki.sei.cmu.edu/confluence/display/c/MSC06-C.+Beware+of+compiler+optimizations*
2. CWE-14: Compiler Removal of Code to Clear Buffers — *https://cwe.mitre.org/data/definitions/14.html*
3. V597. The compiler could delete the `memset` function call (…) — *https://www.viva64.com/en/w/v597/*
4. N1381 — #5 `memset_s()` to clear memory, without fear of removal — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1381.pdf*
5. `openssl/crypto/ec/curve448/utils.c` (old code) — *https://github.com/openssl/openssl/blob/f8385b0fc0215b378b61891582b0579659d0b9f4/crypto/ec/curve448/utils.c*
6. `OPENSSL_cleanse` (implementation) — *https://github.com/openssl/openssl/blob/master/crypto/mem_clr.c*
7. `openssl/crypto/mem.c` (old code) — *https://github.com/openssl/openssl/blob/104ce8a9f02d250dd43c255eb7b8747e81b29422/crypto/mem.c#L143*

8. `openssl/crypto/sparccpuid.S` (example of assembly implementation) — *https://github.com/openssl/openssl/blob/master/crypto/sparccpuid.S#L363*
9. `memzero_explicit` (implementation) — *https://elixir.bootlin.com/linux/v4.18.5/source/lib/string.c#L706*
10. Options Controlling C Dialect — *https://gcc.gnu.org/onlinedocs/gcc/C-Dialect-Options.html*
11. `bzero`, `explicit_bzero` - zero a byte string — *http://man7.org/linux/man-pages/man3/bzero.3.html*
12. `SecureZeroMemory` function — *https://msdn.microsoft.com/en-us/library/windows/desktop/aa366877(v=vs.85).aspx*
13. `memzero_explicit` — *https://www.kernel.org/doc/htmldocs/kernel-api/API-memzero-explicit.html*
14. `OPENSSL_cleanse` — *https://www.openssl.org/docs/man1.1.1/man3/OPENSSL_cleanse.html*
15. Reimplement non-asm `OPENSSL_cleanse()` #455 — *https://github.com/openssl/openssl/pull/455*
16. How to zero a buffer — *http://www.daemonology.net/blog/2014-09-04-how-to-zero-a-buffer.html*
17. Hacker News: How to zero a buffer (daemonology.net) — *https://news.ycombinator.com/item?id=8270136*
18. Mac OS X equivalent of `SecureZeroMemory` / `RtlSecureZeroMemory`? — *https://stackoverflow.com/questions/13299420/*
19. Optimising away `memset()` calls? — *https://gcc.gnu.org/ml/gcc-help/2014-10/msg00047.html*
20. Bug 15495 - dead store pass ignores memory clobbering asm statement — *https://bugs.llvm.org/show_bug.cgi?id=15495*
21. Bug 82041 - `memset` optimized out in `random.c` — *https://bugzilla.kernel.org/show_bug.cgi?id=82041*
22. [PATCH] random: add and use memzero_explicit() for clearing data — *https://lkml.org/lkml/2014/8/25/497*
23. N1548 — C11 draft — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1548.pdf*
24. `memset`, `memset_s` — *https://en.cppreference.com/w/c/string/byte/memset*
25. Test for `memset_s` in gcc 8.2 at Godbolt — *https://godbolt.org/g/M7MyRg*
26. Test for `memset_s` in clang 6.0.0 at Godbolt — *https://godbolt.org/g/ZwbkgY*
27. Test for `memset_s` in MSVC 19 2017 at Godbolt — *https://godbolt.org/g/FtrVJ8*
28. Test for `memset_s` in icc 18.0.0 at Godbolt — *https://godbolt.org/g/vHZNrW*
29. N1967 (WG14) — Field Experience With Annex K - Bounds Checking Interfaces — *http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1967.htm*
30. `SecureString` Class — *https://docs.microsoft.com/en-us/dotnet/api/system.security.securestring*
31. `securemem`: abstraction to an auto scrubbing and const time eq, memory chunk. — *https://hackage.haskell.org/package/securemem*
32. `secstr`: Secure string library for Rust — *https://github.com/myfreeweb/secstr*
33. Sanitizing Sensitive Data: How to get it Right (or at least Less Wrong…) — *https://proteancode.com/wp-content/uploads/2017/06/ae2017_final3_for_web.pdf*
34. Sanitizing Sensitive Data: How to get it Right (or at least Less Wrong…) — *https://link.springer.com/chapter/10.1007%2F978-3-319-60588-3_3*
35. P1315R1 minutes at Kona 2019 — *http://wiki.edg.com/bin/view/Wg21kona2019/P1315*
36. CppCon 2018: Chandler Carruth “Spectre: Secrets, Side-Channels, Sandboxes, and Security” — *https://www.youtube.com/watch?v=_f7O3IfIR2k*
37. The SECURE Project and GCC - GNU Tools Cauldron 2018 — *https://www.youtube.com/watch?v=Lg_jJLtH7R0*
38. The SECURE project and GCC — *https://gcc.gnu.org/wiki/cauldron2018#secure*

39. The Security Enhancing Compilation for Use in Real Environments (SECURE) project — *https://www.embecosm.com/research/secure/*
40. What you get is what you C: Controlling side effects in mainstream C compilers — *https://www.cl.cam.ac.uk/~rja14/Papers/whatyouc.pdf*
41. P1315R1 `secure_val`: a secure-clear-on-move type — *http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1315r1.html*
42. Portable password input — *https://docs.python.org/3.7/library/getpass.html*
43. P1315R2 minutes at Cologne 2019 — *http://wiki.edg.com/bin/view/Wg21cologne2019/P1315*
44. `secure_clear` Example implementation — *https://github.com/ojeda/secure_clear/tree/master/example-implementation*